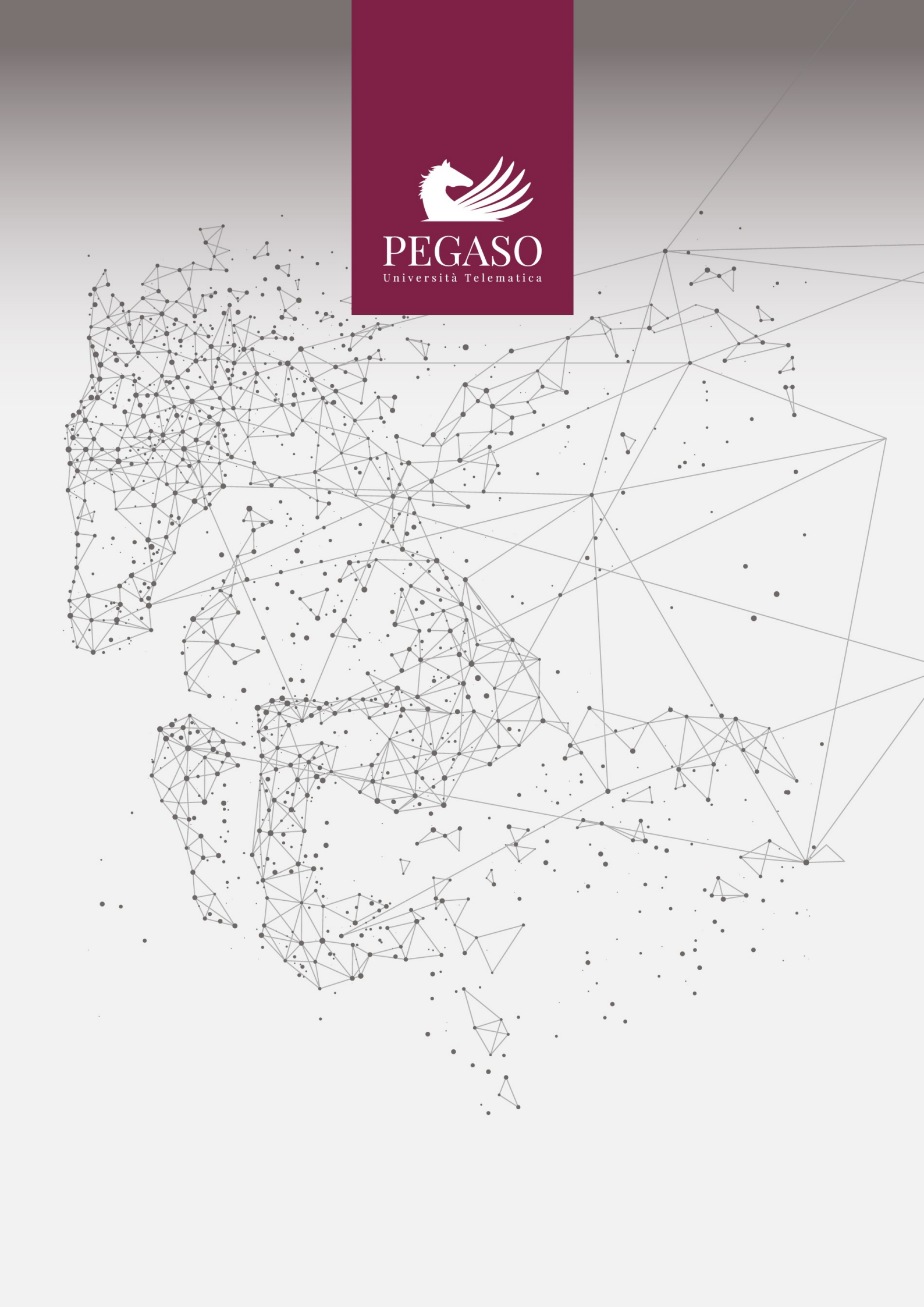

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. ATTRIBUTI E IDENTITÀ 3
2. OPERAZIONI 5
3. INCAPSULAMENTO 7
BIBLIOGRAFIA 15

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Attributi e identità

Un attributo è una proprietà statica di un oggetto e per questo specificato a livello di definizione della classe:

• nome, età, peso sono attributi della classe Persona

• colore, peso, anno, modello sono attributi della classe Auto

forma, peso, colore e tipo di materiale: attributi della classe Soprammobile

I nomi degli attributi devono essere unici all'interno di una classe per evitare ambiguità. Ma se il nome è coerente con il significato semantico della proprietà non può essere altrimenti.

Ogni attributo ha un nome, un valore corrente, un insieme di valori ammessi (dominio). Ad esempio, l'attributo colore della classe Automobile può ha un dominio dei valori {bianco, nero, argento, grigio, blu, rosso, giallo, verde}. Il dominio per l'attributo peso è semplicemente quello dei numeri interi positivi. In situazioni più complesse, il dominio può essere dato a sua volta da istanze di un'altra classe. Ad esempio, sempre la classe Automobile può avere un attributo denominato motorizzazione istanza di una classe Motore, caratterizzato a sua volta da potenza, cavalliFiscali, numeroCilindri e tipoCarburante.

L'insieme degli attributi è comune a tutti gli oggetti di una stessa classe in quanto condividono la stessa semantica essendo realizzazioni concrete di uno stesso concetto, anche se in generale i valori degli attributi saranno in genere differenti nelle diverse istanze; possono anche coincidere ma è un puro caso. Ad esempio, l'attributo età ha il valore "24" nell'oggetto Franco Lorusso della classe Persona ed ha valore "32" nell'oggetto Mario Rossi sempre della classe Persona

L'insieme dei valori degli attributi ad un dato istante di tempo costituisce lo **stato** dell'oggetto. E' possibile assegnare ad un attributo un valore di default assegnato al momento dell'istanziazione dell'oggetto.

L'identità è un'altra proprietà degli oggetti di una classe che si differenzia dagli attributi per il carattere di unicità che presenta.

Nel mondo reale ogni oggetto è unico. Quando si parla di una particolare sedia si sa che si sta parlando della stessa cosa, anche se con passare del tempo alcune sua caratteristiche vengono modificate, ad esempio il costo. Eppure sempre della stessa sedia si parla. In altre parole, la sedia ha un'identità che si conserva nel tempo indipendentemente dalle modifiche che l'oggetto subisce. Viceversa, in un dato momento, oggetti diversi possono avere gli stessi valori per gli attributi ma restano comunque oggetti ben distinti, ad esempio le sedie che si usano ad uno stesso tavolo.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Tutti gli oggetti quindi hanno un'identità e sono distinguibili gli uni dagli altri per la loro esistenza intrinseca e non per le proprietà descrittive che possono avere. Analogamente, gli oggetti del mondo object-oriented hanno un'identità immutabile nel tempo che li distingue e li rende unici indipendentemente dal valore dei suoi attributi che possono anche essere di uguale valore in un determinate momento per oggetti diversi.

Dal punto di vista dell'implementazione, per gestire e disporre dell'identità di un oggetto non è consigliabile definire un attributo id specifico per passarlo come parametro in una chiamata di un'operazione su quell'oggetto. Invece, l'identità è preferibilmente gestita come se fosse un attributo predefinito, unico e immutabile per tutta la vita dell'oggetto realizzato generalmente mediante un codice numerico assegnato direttamente dal sistema al momento della creazione dell'oggetto. Tale codice numerico non può essere letto ma che viene usato per tradurre ogni riferimento all'oggetto. Trattasi di una rappresentazione compatta e usata per far riferimento all'oggetto a basso livello. Definire un attributo id non è più efficiente in quanto non vi è alcun risparmio di memoria, anzi lo spazio usato può essere anche in più avendo definite e fatto uso di un attributo non necessario.

Non bisogna confondere gli identificatori interni con gli attributi del mondo reale. Gli identificatori interni sono definiti per comodità di implementazione e supportati dalla maggior parte dei linguaggi orientati agli oggetti per far riferimento agli oggetti. Ma gli identificatori interni non hanno significato nel dominio del problema. Per esempio, il numero di libretto sanitario o il codice fiscale o il numero di telefono di un oggetto della classe Persona non sono identificatori interni (sono semplicemente attributi).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Operazioni

Un'operazione è un'azione che un oggetto esegue su stimolo di un altro oggetto e che determina una reazione.

Ciascuna operazione incapsulata in un oggetto fornisce una rappresentazione di uno dei possibili comportamenti dell'oggetto espresso tramite la reazione allo stimolo ricevuto. Le operazioni operano sui dati incapsulati dell'oggetto.

Un oggetto, ogni volta che riceve uno stimolo, inizia a fare qualcosa che può essere semplice come recuperare il valore di un attributo (es. il colore di un'automobile), cambiare il valore di un attributo, oppure complesso come eseguire un algoritmo usando anche i dati dei suoi attributi iniziare una reazione a catena di stimuli che sono mandati ad altri oggetti, ad esempio per ottenere delle informazioni per poter eseguire altre elaborazione che possono richiedere l'interazione con altri oggetti diversi. Ad esempio, l'operazione prendiColore per un oggetto della classe Automobile estrarrà e fornirà il colore immagazzinato nell'attributo colore. L'esistenza di questa operazione implica che la classe Automobile è stata progettata in modo che una sua istanza possa ricevere uno stimolo, scatenando l'esecuzione dell'operazione corrispondente, ovvero la lettura del valore dell'attributo colore e la restituzione come valore di ritorno dell'operazione all'oggetto chiamante. Noi chiamaremo questo stimolo un messaggio.

In questo contesto i termini «operazione» e «metodo» sono sinonimi e fin qui li abbiamo usati come tali. Attenzione però in UML i due termini hanno un significato diverso (il metodo è l'implementazione di un'operazione definita per un oggetto usando un linguaggio di programmazione).

In generale tipi di operazioni sono due:

- interrogazione (query): accedono allo stato dell'oggetto senza alterarlo (es. "prendiColore" della classe Automobile)
- modificatore: alterano lo stato di un oggetto (es. "cambiaPeso" della classe Persona)
- esecutore: esegue un algoritmo (es. la radice quadrata del valore di un attributo, la somma peseta fra i valori degli attributi)
-

Inoltre ci sono due operazioni di base per una classe di oggetti (realizzate con modalità diverse a seconda dei linguaggi):

- costruttore: crea un nuovo oggetto e/o inizializza il suo stato
- distruttore: distrugge un oggetto e/o libera il suo stato.

PEGASO
Università Telematica

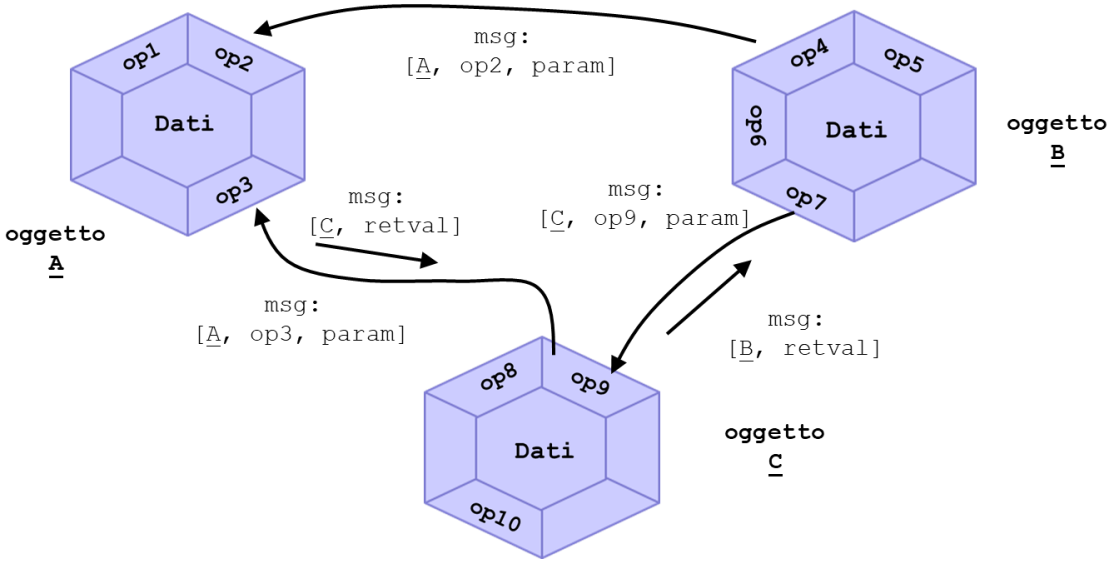
Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Abbiamo detto che l'applicazione con approccio orientato agli oggetti è costruita facendo interagire gli oggetti e per questo scopo nella specifica della classe sono definite una serie di operazioni. Gli oggetto sono costruiti per ricevere stimoli dall'esterno ed eseguire l'operazione corrispondente.

Ma come accade che un oggetto riceva da un altro oggetto uno stimolo per l'esecuzione di un'operazione?

I messaggi sono mezzo con cui gli oggetti interagiscono fra loro, ovvero un messaggio stimola l'oggetto ricevente a fare qualcosa, e precisamente ad eseguire l'operazione associata al messaggio. Il messaggio sarà della forma [oggetto destinatario, operazione, parametri] dove i parametri consistono nelle informazioni che devono essere fornite per eseguire l'operazione.

Come esempio di scambio di messaggi in un sistema object-oriented consideriamo gli oggetti in Fig. 1. Ci sono 3 oggetti, A, B, C che comunicano tra di loro scambiandosi messaggi. Se l'oggetto B richiede il calcolo associato con l'operazione op9 dell'oggetto C, esso manderà a C un messaggio di tipo «msg: [C, op9, param]» specificando quindi l'oggetto destinatario, l'operazione e i parametri di invocazione di quella operazione. Come parte dell'esecuzione di op9, l'oggetto C potrebbe mandare all'oggetto A un messaggio come «msg: [A, op3, param]». Poi A esegue op3 e manda a C un messaggio del tipo «msg: [C, retval] » dove retval è il valore di ritorno della chiamata di op3. Analogamente per il messaggio di ritorno dall'oggetto C a B.



- **Fig. 1: Interazioni e scambi di messaggi fra oggetti**

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Incapsulamento

Un'applicazione con approccio orientata agli oggetti è costruita facendo interagire degli oggetti tramite scambio di messaggi che vanno ad invocare determinate operazioni nell'oggetto. Le operazioni sono utilizzabili (invocabili) con modalità che prescindono completamente dagli aspetti implementativi. Basta sapere come invocarle. Questo è definito dall'interfaccia dell'oggetto specificata a livello di classe.

Il concetto di un'interfaccia è strettamente connesso ad una proprietà chiave dell'approccio orientato agli oggetti: incapsulamento. Infatti l'incapsulamento è un meccanismo atto a limitare l'accesso diretto agli elementi dell'oggetto tramite la definizione di un'interfaccia.

L'incapsulamento è la proprietà per cui un oggetto contiene ("incapsula") al suo interno gli attributi (dati) e i metodi (procedure) che accedono ai dati stessi. Lo scopo principale dell'incapsulamento è dare accesso ai dati incapsulati solo attraverso operazioni predefinite come accessibili dall'esterno in un'apposita interfaccia.

E' come se l'interfaccia formasse un «muro» di astrazioni procedurali con le operazioni definite che rappresentano il solo modo per raggiungere gli attributi ed eventualmente operare su di essi. Poiché le comunicazioni possono avvenire solo attraverso i metodi che costituiscono l'interfaccia che agisce come un "muro" di astrazioni procedurali sui dati, si dice che si realizza l'"information hiding".

Lo scopo è una separazione degli aspetti visibili all'esterno di un oggetto, che sono accessibili ad altri oggetti tramite operazioni definite dall'interfaccia, dai dettagli di implementazione interna dell'oggetto, che sono nascosti agli altri oggetti.

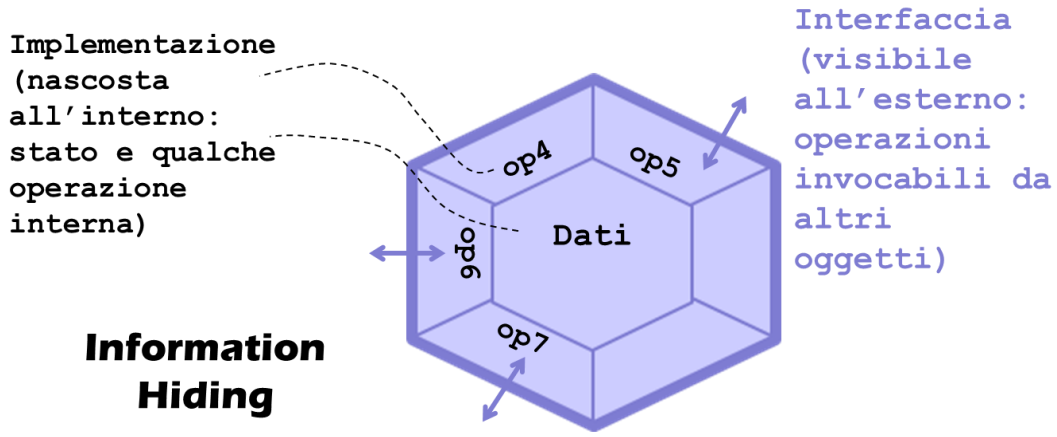


Fig. 2: Incapsulamento, interfaccia e information hiding

Tramite incapsulamento sia le strutture dati che gli algoritmi (elaborazioni interne) possono essere nascosti alla visibilità dall'esterno di un oggetto.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Questo porta a diversi vantaggi.

- Riuso e disaccoppiamento: Le classi che descrivono oggetti ben incapsulati sono facili da riutilizzare in applicazioni differenti. Inoltre è possibile modificare l'implementazione di un oggetto può essere cambiata senza influenzare le applicazioni che lo usano. Quindi si possono modificare le procedure che manipolano i dati di un oggetto senza che queste modifiche si ripercuotano su oggetti esterni che lo usano (fintantoché l'interfaccia rimane invariata).
- Alto grado di modularità: L'incapsulamento può essere anche visto come la suddivisione di un sistema complesso in unità modulari aventi ciascuna una funzionalità ben definita. Con la definizione di un'interfaccia questo permette a queste unità modulari (oggetti) di essere usati in modo indipendente in diverse applicazioni. Inoltre le operazioni di modifica e manutenzione sono molto più facili perché i cambiamenti sono localizzati e non globali, e quindi si riduce notevolmente il rischio di introdurre errori ed il tempo necessario per eseguire le operazioni di test.
- Maggiore protezione dei dati: L'incapsulamento permette di nascondere e non rendere accessibili alcune informazioni (dati e operazioni interne) di un oggetto

L'incapsulamento non è peculiare dei linguaggi orientati agli oggetti, ma in questi la capacità di combinare strutture dei dati e comportamento in una singola entità rende l'incapsulamento più chiaro e più potente che nei linguaggi convenzionali che separano strutture dati e comportamento (come in quelli procedurali).

1. Astrazione

L'astrazione è un altro principio base dell'approccio object oriented, usato come potente mezzo per affrontare la complessità nei problemi da risolvere, in quanto porta a concentrarsi solo su determinati aspetti della realtà che sono rilevanti per il problema.

Ci sono diverse definizioni di astrazione. Una particolarmente significativa è la seguente di Booch: «*Con l'attività di astrazione si individuano le caratteristiche di un oggetto che lo distinguono da tutti gli altri tipi di oggetti fornendo quindi, per l'oggetto in questione, un ben preciso ambito concettuale relativo al punto di vista dell'osservatore*».

Per risolvere un problema, non ci si deve perdere in ogni dettaglio, bensì prestare attenzione solo ai dettagli essenziali nella rappresentazione di un'entità del dominio applicativo del problema. La complessità dei problemi che siamo in grado di risolvere è direttamente correlata alle specie e alle qualità delle astrazioni disponibili.

Dove entra in gioco l'astrazione nella programmazione orientata agli oggetti?

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

L'astrazione nell'approccio orientati agli oggetti entra in gioco nell'individuazione e definizione degli oggetti.

Una classe è frutto di un'astrazione di un dato o più in generale di un concetto del mondo reale.

L'astrazione è quel processo che porta ad astrarre le proprietà rilevanti di un'entità, ignorando i dettagli inessenziali.Dato lo stesso concetto (rilevante nel dominio dell'applicazioni) più astrazioni sono possibili in relazione al problema da risolvere. A seconda del problema dell'utente finale (ovvero dal dominio dell'applicazione) la classe che modella un'automobile può essere definita in due modi differenti. Se si sta scrivendo un'applicazione di vendita (utente finale venditore) l'auto sarà modellata attraverso gli attributi prezzo, durata della garanzia, colore, ...

Se si sta scrivendo un'applicazione per la gestione di un'autofficina (utente finale meccanico) allora l'auto sarà modellata attraverso gli attributi tipo di motore, cilindrata, tipo di olio, ...

Attraverso un processo di astrazione, un programmatore object-oriented modella i problemi reali (fase di abstraction) definendo opportunamente delle classi le cui istanze (gli oggetti) gli permetteranno di risolvere il suo problema.

L'astrazione spinge a concentrarsi sugli aspetti essenziali ed intrinsici di un'entità e nell'ignorare le proprietà secondarie in relazione al problema da risolvere. Nello sviluppo di sistemi, ciò significa concentrarsi su **cosa è e cosa fa un oggetto** prima di decidere come debba essere implementato.

L'uso dell'astrazione dà la libertà di prendere decisioni evitando quanto più a lungo possibile coinvolgimenti prematuri nei dettagli interni o relativi all'implementazione.

L'astrazione è una caratteristica per rendere efficace l'incapsulamento. Infatti porta a prestare maggiore attenzione alla specifica dell'oggetto (interfaccia) piuttosto che sui dettagli implementativi (implementazione delle operazioni). Infatti per usare un oggetto non importa sapere come è fatto dentro. Basta sapere come interagire con esso.

Prendiamo ad esempio, la macchina per il caffè. Ha le forme e colori più disparati, ma tutto quello che bisogna fare per fare una tazzina di caffè è mettere la capsula e premere un pulsante. Quindi pensiamo all'astrazione combinata all'incapsulamento come un piccolo insieme di metodi pubblici che qualsiasi altra classe può chiamare senza "sapere" come funzionano all'interno (se l'acqua della macchina viene riscaldata da una resistenza elettrica o a gas, o con un laser, a noi poco importa, l'importante è che la macchinetta svolga il suo compito).

Il fatto che il focus sia su cosa è e fa un oggetto (sulla specifica dell'oggetto o meglio dell'interfaccia) invece sul come è usato e implementato rende i sistemi costruiti con l'approccio orientato agli oggetti alla lunga più stabile. Infatti, gli utilizzi degli oggetti dipendono pesantemente dai dettagli dell'applicazione e possono cambiare anche frequentemente durante lo sviluppo. All'evolversi delle richieste le caratteristiche

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

fornite da un oggetto sono molto più stabili dei modi con cui è usato. Una classe automobile definita con una certa interfaccia rimarrà tale o subirà pochi cambiamenti rispetto alla logica che la usa.

L'astrazione dei dati viene fornita da tutti i linguaggi di programmazione più moderni, tuttavia nei linguaggi OO insieme all'ereditarietà esprime senza dubbio una potenza maggiore con il risultato di una maggiore chiarezza concettuale con l'astrazione organizzata per livelli nell'ambito di oggetti riconducibili ad una unica matrice di caratteristiche. Ad esempio, cerchio e poligoni sono entrambe figure geometriche e alcune operazioni sono definibili per entrambe in quanto figure geometriche. Questo porta ad un numero minore di casi distinti da capire ed analizzare.

2. Ereditarietà

Partendo dal punto di vista che le classi esprimono astrazione di concetti, è facile comprendere che una classe può essere specializzata da un'altra classe oppure vista dalla prospettiva opposta una classe essere la generalizzazione di un insieme di classi di cui rappresenta un comportamento generale comune.

Ecco, l'ereditarietà è la controparte tecnica (ovvero usata nell'ambito dei linguaggi di programmazione) del concetto di specializzazione/generalizzazione e anche una delle differenze chiave tra i sistemi object-oriented e quelli convenzionali.

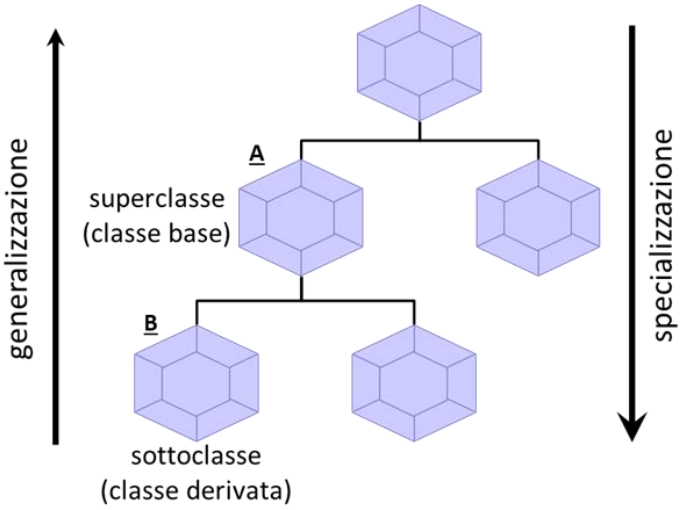


Fig. 3: Ereditarietà come relazione di specializzazione/generalizzazione

L'ereditarietà è una relazione di generalizzazione/specializzazione: la superclasse definisce un concetto generale e la sottoclasse rappresenta una variante specifica di tale concetto generale. Si dice che B è una *sottoclasse* di A e che A è una *superclasse* di B. Denominazioni alternative equivalenti, sono *classe base* per A e *classe derivata* per B

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Inoltre una sottoclasse B rappresentando un raffinamento del concetto rappresentato dalla superclasse, eredita gli attributi e le operazioni della superclasse A salvo poi possono avere ulteriori attributi ed operazioni più specifiche oppure riderifinire le operazioni ereditate (overriding).

In generale, l'uso dell'ereditarietà porta ad una strutturazione e classificazione dei concetti che si esprime attraverso una gerarchia di classi in relazione di specializzazione/generalizzazione tra di loro, cosa che risulta molto utile in fase di modellazione. Infatti, col meccanismo dell'ereditarietà ci si concentra sulla creazione di tassonomie del sistema in esame che offre una notevole chiarezza concettuale rispetto al problema da trattare e quindi la possibilità di soluzioni più efficaci ed efficienti.

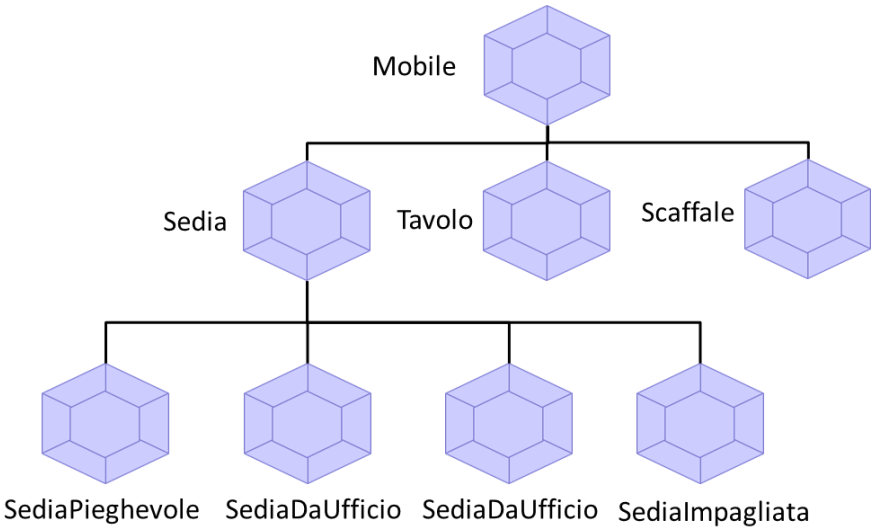


Fig. 4: Ereditarietà ed esempio di gerarchia di classi

Come mostrato in Fig. 4, la classe Sedia potrebbe essere specializzata dalla classe Mobile poiché le sedie sono un particolare tipo di mobile. Analogamente, la classe SediaImpagliata potrebbe essere specializzata dalla classe Sedia e così via in una gerarchia complicata quanto basta per esprimere tutti i concetti rilevanti alla modellazione in corso.

Dato che una sottoclasse rappresenta un raffinamento del concetto rappresentato dalla sua superclassse, gli attributi e le operazioni di una superclasse sono ereditate dalle sottoclassi che poi possono avere ulteriori attribute ed operazioni specifiche. Quindi in modo molto naturale, gli oggetti di una sottoclasse possono essere utilizzati in ogni contesto in cui esse siano trattate da oggetto della superclasse, un quanto un oggetto della sottoclasse come specializzazione è automaticamente anche un oggetto della superclasse gli attribute e le operazioni previste per quest'ultima. Il suo essere oggetto della sottoclasse si esprime andando a ridefinere o ad aggiungere attributi e/o opeazioni nuove.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Ad esempio, se serve un generico oggetto di classe Sedia per potersi sedere, sicuramente un oggetto di classe SediaImpagliata va pure bene bene visto che una sedia impagliata è pur sempre una sedia. Si noti che ovviamente il viceversa non vale, cioè non si possono usare oggetti della superclasse in contesti che si aspettano un oggetto di una sottoclasse, perché potrebbero essere troppo generici e privi delle operazioni e degli attributi necessari; ad esempio non tutte le istanze di Mobile vanno bene per sedersi.

Il meccanismo dell'ereditarietà è di fondamentale importanza nella programmazione ad oggetti, in quanto induce una strutturazione gerarchica nel sistema software da costruire. Questo, unito all'astrazione dei concetti rappresentati dalle classi fornisce una maggiore chiarezza concettuale, software più conciso con un numero minore di casi da distinguere e trattare e quindi software più pulito ed elegante.

Ogni modifica ai dati o alle operazioni contenuti in una superclasse è immediatamente ereditata da tutte le sottoclassi che hanno ereditato da tale superclasse. Pertanto la gerarchia delle classi diventa un meccanismo attraverso cui le modifiche ad alto livello (nelle classi nei livelli alti) possono essere immediatamente propagate a tutto il sistema.

Come già detto, una sottoclasse eredita tutti gli attributi e le operazioni associate alla superclasse. Questo significa che tutte le strutture dati e gli algoritmi originariamente progettati ed implementati per la superclasse sono immediatamente disponibili per la sottoclasse senza fare niente di specifico per ottenerli. Quindi il riuso è stato raggiunto direttamente.

3. Polimorfismo

Un concetto associato all'ereditarietà è il polimorfismo.

Si parla di polimorfismo quando la stessa operazione è implementata in modo diverso da varie classi. Questo in generale. Nel contesto dell'ereditarietà le classi in questione sono sottoclassi che possono implementare uno stessa operazione definita nella classe base in modo differente.

I metodi che vengono ridefiniti in una sottoclasse sono detti *polimorfi*, in quanto lo stesso metodo si comporta diversamente a seconda del tipo di oggetto su cui è invocato.

Nella maggior parte deli linguaggi OO di programmazione, il polimorfismo è particolarmente efficace in relazione al binding. Il binding riguarda la decisione circa l'attributo o il metodo da richiamare in un dato momento dell'esecuzione del programma. Tale decisione può essere stata stabilita in anticipo in maniera fissa in fase di compilazione (in tal caso si parla di **binding statico)**; oppure può essere presa a tempo di esecuzione, in maniera dinamica (in tal caso si parla di **binding dinamico**).

Quindi in presenza di ereditarietà e binding dinamico, con il polimorfismo rende possibile determinare dinamicamente (cioè a tempo d'esecuzione) l'implementazione della operazione specifica da eseguire, in funzione del tipo corrente dell'oggetto overo del tipo più specifico dell'oggetto effettivamente istanziato.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Si consideri a titolo di esempio un'applicazione che deve disegnare tre tipi diversi di grafici, a linee, a torta e istogrammi.

In un'applicazione convenzionale (Fig. 5) la soluzione più ovvia è sviluppare un modulo software di disegno con un sottoprogramma per ogni tipo di grafico che presi i dati di cui fare il grafico produce l'immagine corrispondente. Questo comporta di inserire una logica di controllo per determinare quale operazione invocare ogni volta che si vuole produrre l'immagine di un grafico.

Sebbene questo design sia ragionevolmente immediato, aggiungere un nuovo tipo di grafico potrebbe essere complicato. Occorrerebbe creare oltre ad un nuovo sottoprogramma per disegnare il nuovo tipo di grafico ma soprattutto modificare la logica di controllo per considerare il nuovo tipo di grafico. Quindi l'aggiunta di nuovi tipi di grafico non sarebbe localizzata ai sottoprogrammi che li rappresentano ma avrebbe impatto su altre parti del sistema che devono disegnare il grafico, richiedendo modifiche distribuite in altri punti.

```
case of tipoGrafico:
if tipoGrafico = graficoLinee
        then DisegnaGraficoLinee (dati);
if tipoGrafico = graficoTorta
        then then DisegnaGraficoATorta (dati);
if tipoGrafico = istogramma
        then DisegnaIstogramma (dati);
end case;
```

Fig. 5: Codice procedurale per l'esempio "disegna grafico"

Nel mondo object oriented è possibile un'elegante soluzione (Fig. 6). Basta, infatti, definire tutti i tipi di grafici come sottoclassi di una classe generale detta Grafico avente un'operazione disegna(), che viene ereditata e ridefinita in modo opportuno da ciascuna delle sottoclassi GraficoALinee, GraficoATorta, Istogramma.

Quando si desidera disegnare un grafico, sia esso a torta, a linee o istogramma, basterà invocare ad un oggetto tipoGrafico di classe Grafico il metodo disegna().

Con l'utilizzo del polimorfismo e grazie al dynamic binding, il sistema è in grado di capire autonomamente quale operazione disegna() invocare: sarà quello della sottoclasse coinvolta (ovvero effettivamente istanziata in questo esempio GraficoATorta che è anche di classe Grafico) producendo quindi il tipo di grafico opportuno.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Da notare che l'invocazione dell'operazione disegna() ad un oggetto tipoGrafico resterà sempre identica, indipendentemente da quanti diversi tipi di grafico diversi sono previsti ovvero da quante sottoclassi ci sono e da come l'operazione disegna() sia implementata in ciascuno di esse.

Quando si rende necessario aggiungere un nuovo tipo di grafico al sistema, si crea semplicemente una nuova sottoclasse di Grafico con la sua propria operazione disegna(). Questo corrisponde a produrre nella programmazione convenzionale un nuovo sottoprogramma per il tipo specifico di grafico. Ma, e questo è ancora più importante, in questo nessun cambiamento è necessario all'interno di ogni flusso di esecuzione chiamante qualora si voglia disegnare un grafico salvo istanziare la sottoclasse relativa.

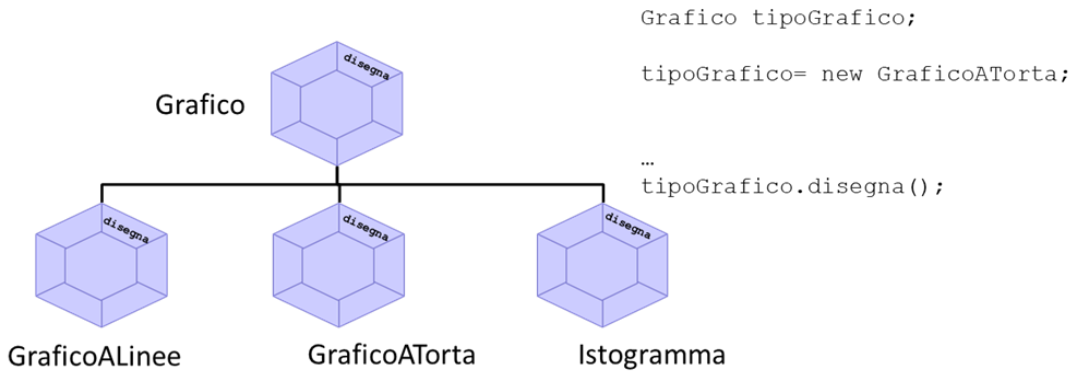


Fig. 6: Polimorfismo nell'esempio "disegna grafico"

Se usato bene, il polimorfismo offre interessanti vantaggi in quanto permette di scrivere codice compatto e di qualità grazie a caratteristiche spiccate di facile estensibilità e robustezza.

Codice facilmente estensibile, in quanto si può indurre il *chiamante* ad invocare nuovi metodi personalizzati includendoli in una classe apposita aggiunta come nuova sottoclasse; nel chiamante basterà istanziare un oggetto di quella classe e invocare il metodo presente nell'interfaccia. Nel chiamante quindi si può utilizzare lo stesso codice personalizzandone o modificandone anche radicalmente il comportamento, senza doverlo riscrivere, ma semplicemente fornendogli *in input* una differente implementazione della sottoclasse e istanziandola.

Software resistente perché eventuali esigenze future nel programma o nella scrittura del codice potranno essere implementate fornendo ad un chiamante già scritto una nuova classe scritta *ad hoc*, quindi con modifiche molto circoscritte e mirate senza interventi sulla logica.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- R. S. Pressman, Principi di Ingegneria del software, McGraw-Hill, Quinta edizione

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).